It seems that they have complicated the problem by a lot. They actually don't need such complex algorithm, but can still achieve very good results.

Spatiotemporal Attention Enhances Lidar-Based Robot Navigation in Dynamic Environments

Jorge de Heuvel Xiangyu Zeng Weixian Shi Tharun Sethuraman Maren Bennewitz

arXiv:2310.19670v2 [cs.RO] 28 Feb 2024

***Abstract*— Foresighted robot navigation in dynamic indoor environments with cost-efficient hardware necessitates the use of a lightweight yet dependable controller. So inferring the scene dynamics from sensor readings without explicit object tracking is a pivotal aspect of foresighted navigation among pedestrians. In this paper, we introduce a spatiotemporal attention pipeline for enhanced navigation based on 2D lidar sensor readings. This pipeline is complemented by a novel lidar-state representation that emphasizes dynamic obstacles over static ones. Subsequently, the attention mechanism enables selective scene perception across both space and time, resulting in improved overall navigation performance within dynamic scenarios. We thoroughly evaluated the approach in different scenarios and simulators, finding excellent generalization to unseen environments. The results demonstrate outstanding performance compared to state-of-the-art methods, thereby enabling the seamless deployment of the learned controller on a real robot.**

I. Introduction

The deployment of mobile service robots around our living areas to improve humans' daily life quality, such as by performing house chores, or carrying out delivery tasks, is an ongoing evolution. For seamless navigation among humans, learning-based navigation controllers represent the forefront of research. A key performance requirement is usually an information-dense representation of the dynamic scene, e.g., with explicitly tracked pedestrians [1]. However, when transitioning away from test and training simulations to the real robot, complex fusion from multiple sensors and hardware-heavy post-processing steps are required to achieve such information-dense dynamics representations [2]–[4]. Here, also feature-rich but costly 3D lidar sensors are appealing [5], [6]. On the other side of the spectrum, many studies focus on learning-based navigation among dynamic obstacles of known position to avoid sensor-based pedestrian tracking [7], [8]. These approaches suffer from a reality gap that hinders generalization to the real world [9], [10]. Following the demand for improved reactive local planners, as recently emphasized by Xiao *et al.* [11], the need for sensor-based lightweight but reliable perception and navigation pipelines emerges that rendundantizes explicit obstacle tracking.

A possible solution is the use of 2D lidar sensors that provide accurate obstacle information within the moving plane of mobile robots [12]. They operate independently of lighting conditions, enabling both day and night operation. But without data such as colors or contours, explicitly tracking objects instances like pedestrians only by their leg profiles from 2D lidar readings is a hard task [13], [14]. Furthermore, the robot's self-movement makes static objects appear dynamic between lidar scans.

All authors are with the Humanoid Robots Lab, University of Bonn, Germany. Maren Bennewitz and Jorge de Heuvel are additionally with the Lamarr Institute for Machine Learning and Artificial Intelligence, Germany. This work has partially been funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under the grant number BE 4420/2-2 (FOR 2535 Anticipating Human Behavior).

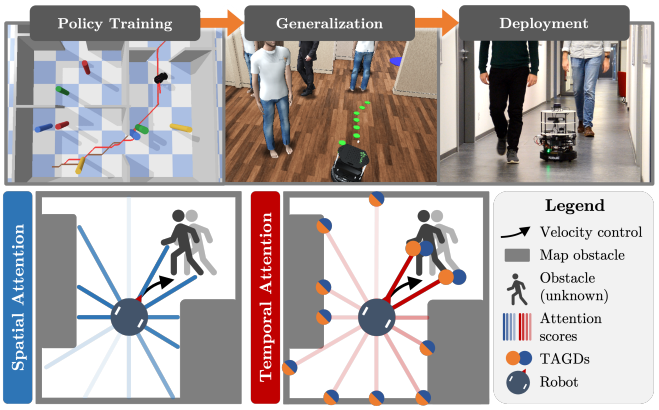


Fig. 1: Our pipeline for learning a robot navigation controller based on lidar. Two attention mechanisms reason about the importance of individual lidar sectors with respect to known and unknown dynamic obstacles. Our Temporal Accumulation Group Descriptors (TAGD) reveal moving obstacles from subsequent lidar scans affected by robot self-motion.

While most current methods leverage convolutional neural networks (CNNs) to process and extract features from lidar data [15], [16], a recent appealing idea to tackle these sensor-implicit obstacle representations is selective attention on a collision-relevant subsectors of the lidar data [17]. Especially when a temporal observation sequence provides dynamic scene information, selective attention on moving obstacles can be beneficial.

To address this, we introduce a novel feature extraction technique tailored for 2D point clouds, incorporating both spatial and temporal attention across the sensor readings. This approach distills critical navigational information, offering a more robust solution for learning-based navigation in dynamic indoor environments. We demonstrate better than state-of-the-art generalization to unseen navigation scenarios and enable a smooth sim-to-real transfer of the learned policy, as we will be able to demonstrate in the experiments.

In summary, the main contributions of our work are:

- A deep reinforcement learning-based (DRL) navigation controller that learns dynamic obstacle avoidance implicitly from 2D lidar readings only.
- A spatiotemporal attention module that infers the relative importance of different observation sectors with respect to proximity and obstacle motion trends.
- A novel 2D lidar observation representation highlighting dynamic obstacles over the robot's self-motion called temporal accumulation group descriptor (TAGD).

II. Related Work

Where mobile robot navigation decomposes into global path planning and local obstacle avoidance, the latter can be tackled with traditional and learning-based approaches. While traditional approaches such as the popular dynamic window approach (DWA) [18], [19] have been advanced with motion prediction [20], they come with the difficulties to avoid C-shaped or dynamic obstacles, or the necessity for re-tuning in different environments [21].

A. Learning-based navigation

Deep learning-based methods [22] appeal with decent generalization performance and less tedious fine-tuning as compared to hard-coded controllers.

Especially reinforcement learning-based (RL) methods have successfully been applied to motion planning [12], [17], [23]–[25]. These works however do not embed dynamic scene understanding, thus limiting the agent's capability around walking pedestrians. Methods like SARL [7] or MP-RGL [8] capture interactions between robot and humans with excellent results, but rely on the known velocity of humans. Others infer or forecast human behavior by predicting their long-term goals, or by predicting their future motion and activities [26], often by employing 3D lidar or RGB(D) cameras [16], [27], [28]. The challenge arises with our aim to learn time-series motion trends for scene-dynamics aware navigation from 2D lidar readings in an end-to-end manner.

B. Point Cloud Feature Extraction

For the feature extraction in a deep RL navigation task, the spatial nature of lidar point cloud data suggests convolutional neural networks (CNN) as natural fit [15], [16]. Here, reducing the input dimensionality into a sparse encoding is a pivotal step. Taking into account the temporal dimension for scene dynamics understanding, individual lidar scans may be CNN-processed separately, followed by a multilayer perceptron (MLP) for joint extraction [16]. With PointNet [29], a high performing network architecture for 3D point cloud registration has been proposed that was recently put to test in a short-horizon RL-based robotic manipulation task [30]. For obstacle pose and dynamics estimation, using a point cloud segmentation approach represents a viable avenue [6]. Looking at the non-learning-based domain, obstacle tracking from point cloud data has been presented before [31], [32]. With the advent of transformer models, the self-attention operator's invariance to cardinality and permutation of input data has proven to be a useful property for point cloud inference [33]. Building on this foundation, our work leverages attention-based 2D lidar feature extraction. This approach enhances deep RL-based local-obstacle avoidance, while integrating high-level guidance from a conventional path planner.

III. Problem Statement and Assumptions

In this work, we consider a differential-wheeled robot pursuing a global goal in a cluttered and dynamic indoor environment, compare Fig. 3a). A map of the empty environment is available for global path planning via A*. Static or dynamic pedestrians however are unknown obstacles to the robot. Also, the pedestrians at different speeds move rigorously without avoiding the robot in their motion, in contrast to other social navigation studies [15]. Therefore, smart and foresighted local collision avoidance is entirely up to the robot. The controlling agent has access to subsequent 2D lidar readings and upcoming path waypoints as observations, which it maps to linear and angular velocity commands. We formulate the task as in a learning-based manner and apply off-policy DRL. In summary, the proposed controller should be able to achieve two tasks: 1) Pursue the global goal through guidance of the computed path and 2) effectively avoid dynamic obstacles on a local scale.

IV. Our Approach

This section explains our novel temporal accumulation group descriptor for lidar readings and subsequently the learning framework.

A. Temporal Accumulation Group Descriptor (TAGD)

It is inherently difficult to capture motion trends of moving obstacles from consecutive 2D lidar readings when the robot is in motion. To reveal moving obstacles over static ones without explicit obstacle tracking, we introduce our novel TAGD. We assume lidar scans to be recorded at a constant frequency of $1/\Delta t$ with a range of $d_{\max}$. Our approach is described in Algorithm 1 with a visualization of all major steps is shown in Fig. 2. We start with the min-pooled 2D lidar points $\mathcal{B}_{t-1}, \mathcal{B}_t$ with N points each. To eliminate the impact of robot rotation and translation, ICP [34] aligns $\mathcal{B}_{t-1}$ to $\mathcal{B}_t$ in the transformed point set $\mathcal{B}'_{t-1}$ (Fig. 2.1). For static obstacles, the points now match up while their positions misalign for dynamic obstacles (Fig. 2.2). For spatial clustering and subsequent temporal matching, clustering group centers g^i are formed along N_c uniformly cast rays by determining the robot-closest point within an angular threshold θ_{thresh} (Fig. 2.3). For temporal matching and dimensionality reduction, the points in $\mathcal{B}'_{t-1}$ and $\mathcal{B}_t$ are assigned to clustering groups $\mathcal{G}^i_{t-1}$ and $\mathcal{G}^i_t$. This assignment is based on the Euclidean distance to their clustering center $\mathbf{g}^i$, within a fixed threshold $d_{\text{thresh}} = 0.25\text{m}$ (Fig. 2.4). Note that d_{thresh} is a static parameter and chosen with a safety margin based on the relation between maximum expected obstacle speed and the inference time step as $d_{\text{thresh}} > v_{\max}\Delta t$. The 2D centroids $\mathbf{c}^i$ of each group $\mathcal{G}^i_{t-1}$ and $\mathcal{G}^i_t$ counteract sensor noise and finally represent a single TAGD $(\mathbf{c}^i_t, \mathbf{c}^i_{t-1})$ (Fig. 2.5).

A TAGD represents the center of data points across two consecutive lidar scans close to the nearest obstacle within an angular zone, such that even small obstacles hit by only a single ray are successfully represented by a TAGD. Note that it is possible for a single dynamic obstacle to be represented in more than one TAGD, depending on the positions of clustering centroids, e.g., see TAGDs 26 and 27 in Fig. 2.4). However, such double representations did not hinder the performance in context of the learned controller. With regards to real-world pedestrians and their leg motion pattern, the influence of faster-than-body moving single legs on the TAGD displacement and therefore body speed estimation

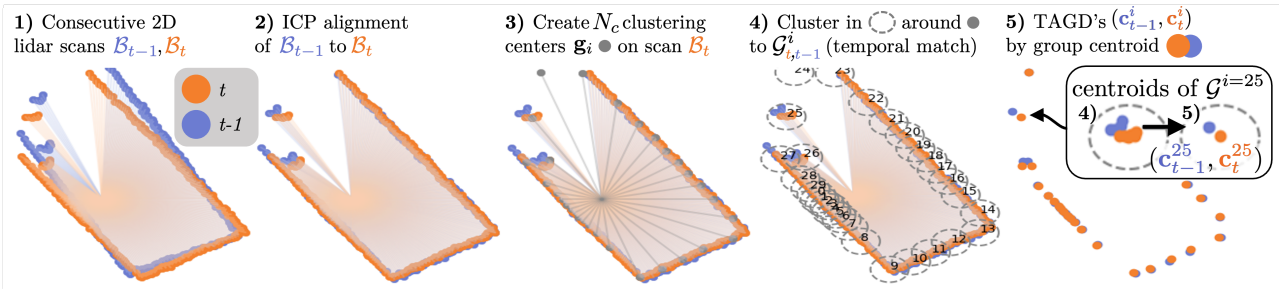


Fig. 2: Schematic of the TAGD generation process. The ICP alignment of two subsequent lidar scans (1) in 2D Cartesian coordinates reduces the effect of robot self-movement (2). This allows better differentiation between dynamic obstacles and static obstacles. The aligned scan is grouped and clustered around ray-cast centers (3). From the clustered points (4), the position difference of the centroid from both time steps reveals a moving obstacle (5).

cannot be entirely ruled out. However, the group centroid calculation within d_{thresh} supports averaging out the effects of sensor noise or displacement and speed of individual legs, even though leg walking patterns are not explicitly considered or simulated in this work. It is worth noticing that a consistent inference timing between the lidar scans is key to correctly represent a given obstacle velocity with TAGDs, also with regards to a sim-to-real transfer. Here, this is directly based on the reinforcement learning control time step $\Delta t = 0.2\text{s}$. We have not used odometry or IMU data for enhanced ICP alignment, but solely rely on the observed static obstacles in the scene. While posing a limitation, this is a defensible assumption for indoor environments. However, we will evaluate the reliance and performance dependency of the navigating RL agent on correct ICP alignment in two ways, 1) without static obstacles in open space among dynamic obstacles, and 2) with ICP alignment artificially turned off. In summary, TAGDs reveal obstacle motion and will therefore be used as input to the temporal attention module of our pipeline.

Algorithm 1 Temporal Accumulation Group Descriptors

Require: Lidar readings $\mathcal{B}_{t-1}, \mathcal{B}_t$, $d_{\text{thresh}}, d_{\max}, N_c$
  $\theta_{\text{thresh}} \leftarrow \pi / N_c$
  $\mathcal{B}'_{t-1} \leftarrow \text{ICP}(\mathcal{B}_{t-1}, \mathcal{B}_t)$
  Initialize TAGD list $\mathcal{C}_t = \{\}$
  for $i = 0$ to N_c **do**
    $\theta_{\text{ref}} \leftarrow 2\pi i / N_c$
    $T \leftarrow \{\mathbf{b} = (r, \theta) \in \mathcal{B}_t \mid |\theta - \theta_{\text{ref}}| \leq \theta_{\text{thresh}}\}$
    $r_{\min} \leftarrow \min_{(r,\theta)\in T}(r, d_{\max})$
    $\mathbf{g}_i \leftarrow (r_{\min}, \theta_{\text{ref}})$
    $\mathcal{G}^i_t \leftarrow \{\mathbf{b} \in \mathcal{B}_t \,|\, \text{dist}(\mathbf{b}, \mathbf{g}_i) \leq d_{\text{thresh}}\}$
    $\mathcal{G}^i_{t-1} \leftarrow \{\mathbf{b}' \in \mathcal{B}'_{t-1} \,|\, \text{dist}(\mathbf{b}', \mathbf{g}_i) \leq d_{\text{thresh}}\}$
    TAGD $(\mathbf{c}^i_t, \mathbf{c}^i_{t-1}) \leftarrow (\text{centroid}(\mathcal{G}^i_t), \text{centroid}(\mathcal{G}^i_{t-1}))$
    $\mathcal{C}_t \leftarrow \mathcal{C}_t \cup \{(\mathbf{c}^i_t, \mathbf{c}^i_{t-1})\}$

B. Deep Reinforcement Learning for Navigation

We choose a deep deterministic policy gradient (DDPG) architecture consisting of an actor and a critic, modeled by neural networks [35]. DDPG features a continuous action space, allowing for smooth robot control. The actor network outputs linear and angular velocities for the robot. The RL framework is based on the Markov Decision Process: An agent in state s_t at time step t decides upon an action a_t based on a policy $\pi(s_t) = a_t$. Upon reaching the next state s_{t+1}, it receives a reward r_t. The optimization objective is the maximization of the γ-discounted cumulative return $R = \sum_{i=t}^{T} \gamma^{i-t} r_t$, where $\gamma = 0.98$. As DDPG is an off-policy RL algorithm, the state-action pairs are stored in a experience replay buffer of length $N_{\text{RP}} = 2,000,000$ and sampled in batches for policy updates.

C. State and Action Space

The state space defines the observations we provide to the agent. As can be seen in Fig. 3b, the agent has access to 2D lidar sensor data, the lidar-derived TAGDs, and upcoming waypoints for global guidance: The $N = 180$ ray min-pooled lidar scan $\mathcal{B}_t$ is represented as a set of robot-centric Cartesian 2D points. Focusing on local obstacles, the lidar scanning range is limited to $d_{\max} = 3.5\text{m}$. To sense dynamic obstacles, we provide the TAGDs $\mathcal{C}_t = \{(\mathbf{c}^i_t, \mathbf{c}^i_{t-1}) | 0 \leq i < N_c\}$ as described earlier, where the number of TAGD's $N_c = 30$ is equal to the number of spatial sectors $N_b = 30$. The value $N_c = 30$ was heuristically chosen so that the clustering groups would jointly provide a cohesive circular coverage mid-range of the lidar distance around $d = 2.5\text{m}$, and the TAGD clustering circles defined by the radius d_{thresh} start to overlap ($N_c \approx 2\pi d / (2 d_{\text{thresh}})$). From the robot nearest waypoint $\mathbf{p}^c$ on path $\mathcal{P}$, we sample $N_f = 5$ waypoints spaced at $\Delta \mathbf{p}^i = 0.3\text{m}$ towards the goal. These are converted to robot-centric Cartesian coordinates and input to the agent as $\mathcal{P}^f_t = \{\mathbf{p}^i | c \leq i < c + N_f\}$.

The continuous action space of the agent consists of linear and angular velocities (v, w), with a range of $v \in [0, 1.0]\text{m s}^{-1}$ and $w \in [-\pi, \pi]\text{rad s}^{-1}$. The robot is not allowed to drive backwards to foster foresighted navigation.

D. Reward

The navigating agent's overall objective is to navigate collision-free along a given path among unknown dynamic obstacles. The reward r_t is therefore a weighted sum:

$$r_t = \alpha_1 r^{\text{collision}}_t + \alpha_2 r^{\text{guide}}_t + \alpha_3 r^{\text{prox}}_t \quad (1)$$

$\alpha_1 = 10$, $\alpha_2 = 0.2$ and $\alpha_3 = 3$ are the experimentally determined weighting factors.

To encourage collision-free navigation, we penalize with $r^{\text{collision}} = -1$ upon collision of the robot with any obstacles.

A natural guidance along the global path is beneficial as it encourages the agent to drive towards the goal. From the current closest waypoint $\mathbf{p}^c$ on the path to the robot, we interpolate 0.6m forward along the path to obtain the guidance point $\mathbf{p}^g$. The distance between $\mathbf{p}^g$ and the robot's position $\mathbf{p}^r$ are penalized with $r^{\text{guide}} = -||\mathbf{p}^g - \mathbf{p}^r||$. By

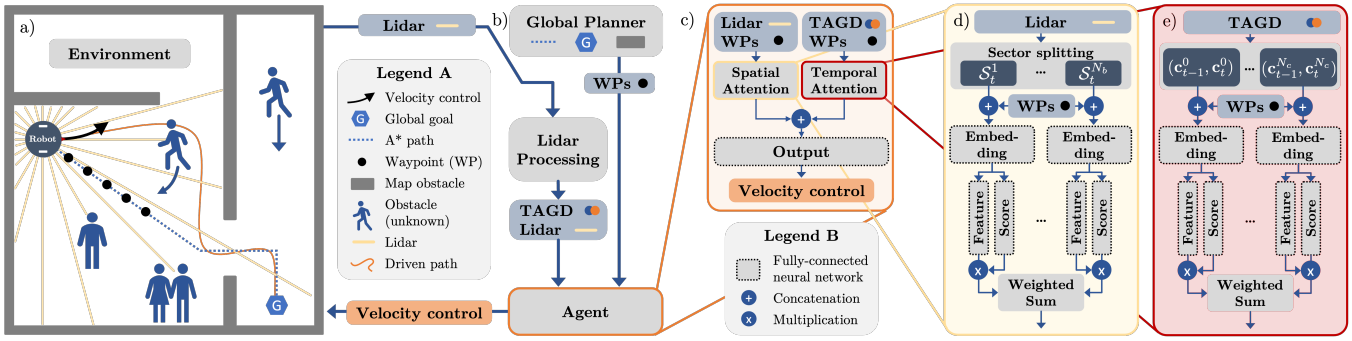


Fig. 3: Illustration of our architecture. **a)** The indoor environment provides lidar readings to the deep reinforcement learning agent that drives a differential-wheeled robot via linear and angular velocity commands. **b)** From subsequent lidar readings, the TAGDs are computed. Merged with the five upcoming waypoints of the global path and the raw lidar readings as observations, they are **c)** processed by the agent in a separate spatial and temporal stream. Both streams feature an attention block to weigh the importance of **d)** individual lidar sectors (spatial) or **e)** the TAGDs (temporal), with respect to the upcoming waypoints. After feature extraction, both streams are concatenated for further processing in the final output network of the actor-critic agent.

design and due to the update at every time step, $\mathbf{p}^g$ cannot be reached, thus providing a continuous penalty that increases when the robot deviates from the path and encourages the robot to drive back to the path in a forward-leading manner.

The concept of r^{prox} aligns with the sparse collision reward, but does not terminate the episode for easier learning. Instead it alerts the agent in vicinity to obstacles about higher risk of collisions, or in other words encourages the agent to keep clear of obstacles. When the minimum distance d between robot and any lidar-scanned obstacle falls below $d_{\text{prox}} = 0.5\text{m}$, a linearly growing penalty is computed as $r^{\text{prox}} = -1 \times |d_{\text{prox}} - \min(d, d_{\text{prox}})|$, else $r^{\text{prox}} = 0$.

E. Network Architecture

As shown in Fig. 3c, our agent's architecture is constructed around two data streams. The individual streams extract spatial and temporal features via an attention mechanism, respectively. Note that both the down-sampled lidar input of the spatial, and the TAGD input of the temporal steam contain partially redundant information due to their origin in the raw distance readings.

1) Temporal and Spatial Data Stream: With the individual TAGDs and the possibly attention-relevant, therefore redundantly-represented upcoming waypoints, we construct N_c individual vectors $\mathbf{U}_{temp} = \{[\mathbf{c}^i_{t-1}, \mathbf{c}^i_t, \mathcal{P}^f_t] | 0 \leq i < N_c\}$, to be passed on to the temporal attention module as $\mathbf{y}_{temp} = Att_{temp}(\mathbf{U}_{temp})$, see Fig. 3e. In the spatial data stream, the lidar scan $\mathcal{B}_t$ of N rays at time step t is split into N_b angular sectors $\mathcal{S}^i_t = \{\mathbf{b}^j_t \in \mathcal{B}_t | iN_b \leq j < (i+1)N_b\}$ with N/N_b rays each. Again, each sector-vector is concatenated with next path segment forming N_b individual vectors $\mathbf{U}_{spat} = \{[\mathcal{S}^i_t, \mathcal{P}^f_t] | 0 \leq i < N_b\}$, jointly passed on to the spatial attention module as $\mathbf{y}_{spat} = Att_{spat}(\mathbf{U}_{spat})$. After both data streams have been processed by their attention modules, respectively, they are concatenated and jointly processed by an output module $O(\cdot)$ for joint feature extraction $\mathbf{o} = O(\mathbf{y}_{temp}, \mathbf{y}_{spat})$. Two separate modules of this pipeline form the actor and critic.

2) Attention module: Both temporal and spatial attention modules $Att(\cdot)$ share a similar network architecture, but no parameters. A visualization of our lightweight attention module can be found in Fig. 3d-e. It is constructed with an embedding, a score and a feature network, inspired by Chen *et al.* [7] and [17]. The embedding module $E(\cdot)$ encodes the input vectors individually along the attention dimension to $\mathbf{e}_i = E(\mathbf{u}_i)$. The embedding $\mathbf{e}_i$ is fed into the score module $S(\cdot)$ that outputs the attention scores $\mathbf{s}_i = S(\mathbf{e}_i)$. All attention scores are Softmax-normalized to obtain the final importance weight. In parallel the embedding is also fed into the feature module $F(\cdot)$ that generates the feature representations as $\mathbf{f}_i = F(\mathbf{e}_i)$. Finally, the feature vectors are scaled by their importance in a weighted sum.

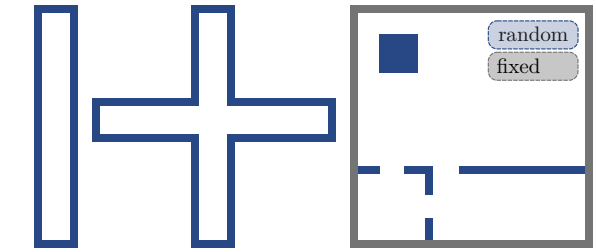
$$\mathbf{y} = Att(\mathbf{U}) = \textstyle\sum_i \text{Softmax}(\mathbf{s}_i) \cdot \mathbf{f}_i \quad (2)$$

$$= \textstyle\sum_i \text{Softmax}(S(\mathbf{e}_i)) \cdot F(\mathbf{e}_i) \quad (3)$$

Note that the due to the lightweight implementation of our attention scheme, the dimensionality along the attention axis reduces from N_b or N_c vectors to one in the output. In other words, the individually embedded lidar sectors or TAGDs do not attend to each other, but the attention scales their impact in the weighted sum, respectively. This form of attention is also referred to as *location-based* attention [36], [37]. All networks describe above are constructed with as ReLU-activated multi-layer perceptrons (MLP)[1].

F. Indoor Training Environments

To train our navigation agent, we use the Pybullet [38] physics engine. We use the minimalistic but well-randomizing indoor environments of de Heuvel *et al.* [17] featuring dynamic cuboid obstacles that represent pedestrians, with three different types of scenarios, see Fig. 4: Corridors, intersections, and offices. The randomization of



a) Corridor b) Intersection c) Office

Fig. 4: The Pybullet-based environments of [17] are used for training. **a)** In the corridor and **b)** intersection environment, the wall distances are randomized (blue). **c)** In the office environment, the outer walls are fixed with randomized inner wall placement for diverse room setups.

[1]Layer sizes (hidden nodes): embedding: $256 \times 128 \times 64$, score: $60 \times 50 \times 1$, feature: $80 \times 50 \times 30$, output: $128 \times 64 \times 64 \times \{1, 2\}$ (critic/actor)

wall density and placement provides varying levels of scene complexity. The corridor environment is long and narrow with a length between $[6\text{m}, 8\text{m}]$ and a width between $[2.0\text{m}, 2.5\text{m}]$. The robot encounters pedestrians moving in opposite directions. The intersection environment is cross-shaped featuring hallway widths between 2.0m and 2.5m, and includes corners that create blind spots for sudden pedestrian appearances. The office environment features a fixed outer size with randomized interconnected rooms and introduces doorway encounters where the robot waits for pedestrian clearance before proceeding. Our room types cover typical encounters suggested for social navigation tasks [39], as found also in other related studies [15], [16]. While our rectangular environments generate variety through architectural randomization, other works achieve variety through larger but static, non-rectilinear scenes [28]. The robot's start and goal location are sampled in the corners or dead ends of the scenes, respectively.

1) Obstacle simulation: Dynamic and static pedestrians represented by cuboids move back and forth through the environments along A* paths with randomized quantity ($N_{\text{dyn}} \in [1, N_{\text{dyn}}^{\text{max}}]$, $N_{\text{stat}} \in [1, 2]$), speed ($v_{\text{ped}} \in [0.5, 1.0]\text{m}\,\text{s}^{-1}$), start, and goal position. Note that the pedestrian speed can exceed the robot's maximum velocity. The maximum dynamic obstacle number $N_{\text{dyn}}^{\text{max}} \in \{2, 4, 8\}$ follows a curriculum scheme (three levels) and is increased over the course of training, whenever the evaluation success rate exceeds 70%. For the purpose of increasing the obstacle encounter likelihood with the robot, start and goal locations of the first pedestrians are sampled around the robot path. All other pedestrians will cross the robot path eventually. Note that the A*-following pedestrians do not take into account each other or the robot position, but rigorously move forward. Collision avoidance is therefore entirely up to the robot, similar to [16], [28] This can lead to highly challenging navigation encounters, especially for larger obstacle numbers. This is in contrast to other studies [15] that simulate the pedestrians motion based on Optimal Reciprocal Collision Avoidance (ORCA), where the pedestrians avoid each other. Notably, also the robot is actively avoided by the pedestrians, easing the collision-free navigation task for the RL agent. Other works have employed the social force model for crowd navigation [40]. Though our more basal dynamic obstacle simulation leads to occasional pedestrian mesh overlaps and occasionally non-passable situations, our selection of an only path-based model is justified by our study's primary focus on feature extraction for RL-driven dynamic obstacle avoidance, rather than on crowd navigation.

G. Robot Model

We employ a differential-wheeled robot, more precisely, the Kobuki Turtlebot 2. The Turtlebot performs angular turns with a speed difference between both wheels. A Slamtec RPlidar A3 2D lidar sensor is mounted on top of the Turtlebot, emitting 1,440 beams. In simulation, we add sensor noise to the distance readings with an amplitude of 2.5cm.

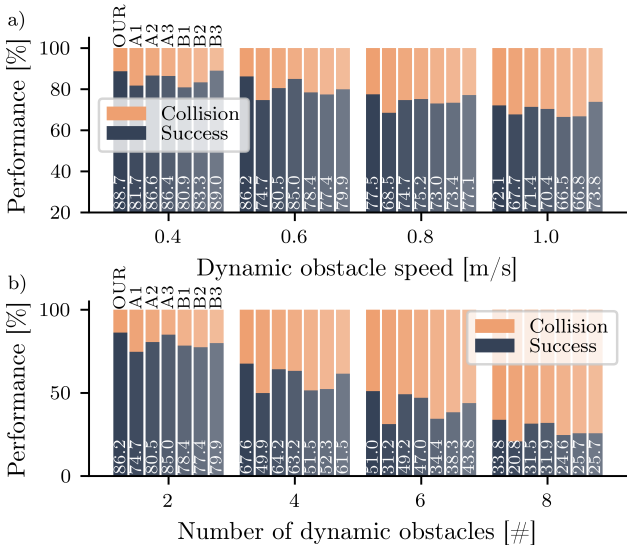


Fig. 5: Performance overview for all approaches averaged over 1,000 episodes with identical scene setups in all three Pybullet environments for **a)** increasing obstacle speeds, with two dynamic and one static pedestrians, and **b)** increasing number of obstacles, with a fixed pedestrian speed $0.6\text{m}\,\text{s}^{-1}$.

V. Experiments

In the following we present the training and evaluation details, followed by an ablation and baseline study. After evaluating the domain shift to the iGibson simulator, the section is rounded up by the real-robot deployment.

A. Training Setup

An episode denotes one navigation run of the robot from start until one of the termination criteria is reached: Collision with other obstacles, timeout after $T_{\text{timeout}} = 150 \equiv 30\text{s}$ steps, or goal-reaching upon vicinity of 0.2m to the global goal. To foster generalization abilities, for each episode a randomly generated environment is setup, as described in Sec.IV-F. The inference and control time step of the agent is set to $\Delta t = 0.2\text{s}$, which also represents the time difference between subsequent lidar scans for the temporal processing. The learning rates for both actor and critic is 1×10^{-4}. All agents presented are trained for 300,000 episodes and evaluated regularly. The best performing model checkpoint of the highest curriculum level is selected for all approaches.

B. Quantitative Performance

We evaluated our trained models with respect to success rate, collision rate, timeout rate, and navigation time over 1,000 episodes. For comparability, the 1,000 episodes were setup identically among all approaches. The flagship approach presented in this study is denoted with OUR. Generally, with challenging environment complexity due to increased obstacle velocities (Fig. 5a), or increased number of dynamic obstacles (Fig. 5b), the success rate stagnates.

1) Ablation Study: We did an ablation study with respect to OUR approach described above to evaluate the contribution of each module to the results, see Tab. Ia) and Fig. 5.

A1 NO-SPATIAL: As OUR, but removing the spatial attention stream, leaving only TAGD and waypoint processing.

A2 NO-TEMPORAL: As OUR, but with no temporal stream or TAGD input, leaving only the spatial single time step attention stream and waypoint processing.

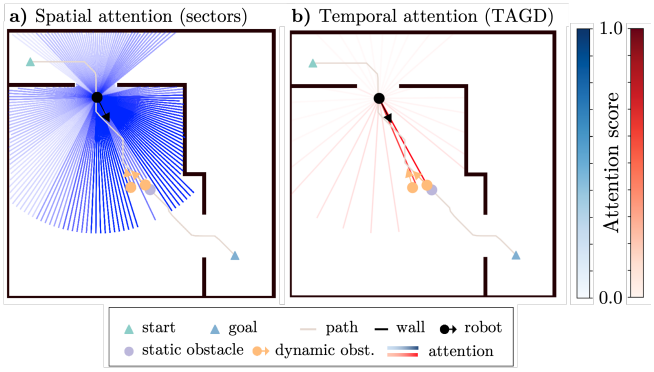


Fig. 6: Exemplary visualization of the **a)** spatial (blue) and **b)** temporal attention (red) for a given navigation scene. The attention scores were color-mapped onto the lidar beam sectors for the spatial and on the beams pointing towards the TAGDs for the temporal attention, respectively. Increased spatial attention towards the forward-facing lidar sectors, as well as increased temporal attention towards the oncoming dynamic obstacle can be observed.

A3 NO-TAGD: As OUR, but without TAGD preprocessing. The network structure implements the spatial attention stream twice with separate network parameters, each processing one of the consecutive lidar scans, respectively.

As can be seen from Tab. Ia) and Fig. 5, with all ablations the performance deteriorates. The joint contribution of spatial and temporal attention emerges with A2 NO-TEMPORAL having a lower success rate compared to OUR, as it relies only on single-time step spatial information.

C. Baselines

To identify the contribution of our feature extraction approach, we compared against two baseline architectures. All baselines leverage 2D lidar (360°) for learning-based mobile robot navigation and were trained in the same environment and training parameters as our approach. The baseline-related modifications lie in the state space content and processing network architectures.

1) Liang et al. - *B1:* A highly-related state-of-the-art approach has been presented in [16]. Similarly to ours, it is an end-to-end obstacle avoidance algorithm originally trained with Proximal Policy Optimization. The authors use 2D lidar and a depth camera to perceive the environment, while the controller outputs velocity commands. From both perception modalities, we solely implement the lidar-related preprocessing and network architecture to replace our attention blocks, which is a 1D CNN taking in three consecutive scans. Precisely, this module is composed of two 1D CNN layers followed by a fully-connected MLP. In contrast to our approach with 2D Cartesian point lidar representation, single-value lidar distance readings are used. The state space still contains five upcoming waypoints, which in contrast to OUR are processed by a separate MLP. Without convergence and therefore not included, we have also tested a closer-to-the-original implementation (512 lidar rays, no waypoints, only goal position).

a)	**Ablation**	SR↑	CR↓	TR↓	Nav. time↓
	OUR	**86.2**	**13.8**	0.0	**17.7** s
	A1: NO-SPATIAL	74.7	25.3	0.0	18.1 s
	A2: NO-TEMPORAL	80.5	19.5	0.0	17.9 s
	A3: NO-TAGD	85.0	15.0	0.0	17.8 s
b)	**Baseline**				
	B1: Liang *et al.* [16]	78.4	21.6	0.0	18.9 s
	B2: Pérez-D. *et al.* [15]	77.4	22.6	0.0	18.6 s
	B3: Pérez-D. *et al.* [15]	79.9	20.1	0.0	18.4 s
c)	**Generalization**				
	iGibson [41]	79.2	18.6	2.2	19.0 s

TABLE I: Performance rates in [%] with respect to success (SR), collision (CR), and timeout (TR) and average navigation times for successful episodes of **a)** ablation and **b)** baseline study averaged over 1,000 episodes, with 2 dynamic pedestrians ($0.6\mathrm{m\,s^{-1}}$) and 1 static pedestrian. The **c)** generalization evaluation reveals slightly decreased performance for the post-training domain shift to the iGibson simulator on similar navigation tasks in more complex environments.

2) Pérez-D'Arpino et al. - *B2/3:* In the end-to-end lidar navigation approach of [15], no temporal information but only the current lidar reading is processed. Similar to Liang *et al.* [16], the authors employ a lidar-processing 1D CNN but with three layers followed by a fully-connected layer. Furthermore, $N = 128$ single-value lidar distance readings are used. Additionally, the global goal position and next upcoming waypoints of the A* path ($\Delta\mathbf{p}^i = 1.0\mathrm{m}$) are part of state space. In B2, we employ their state space and replace our attention block with their lidar-processing CNN and waypoint-processing MLP modules. A sub-version (B3) uses only their CNN architecture but our original state space with regards to waypoints and lidar resolution.

3) de Heuvel et al. *:* In initial tests, we compared against [17], outputting collision-free subgoals instead of velocity commands from single-time step lidar data with similar spatial attention. Direct comparisons with our current method are not viable due to later changes in the training settings. Despite this, the comparison showed a 5.3% performance boost by incorporating TAGDs and temporal attention, motivating our current work.

As can be seen in Tab. Ib) and Fig. 5, for our setup, the CNN-related baselines B1-B3 struggle more with increased number of obstacles. In almost all cases, our approach outperforms all baselines in terms of success rate.

D. Qualitative Attention Analysis

Fig. 6 visualizes the learned spatial (a) and temporal (b) attention for a given navigation scenario. Here, two dynamic obstacles approach the robot from opposite directions, the robot has just entered the room. The spatial attention highlights the forward lidar sectors in the desired direction of navigation. The robot navigates along a wall that locates on its left hand side and we can observe an increased attention on the corresponding lidar sectors. Intuitive to the human eye, the temporal attention focuses the TAGDs of the oncoming dynamic obstacles. Similar to the spatial attention, a slightly increased temporal attention can be observed in forward direction of the robot. In direct comparison to the temporal stream, the spatial stream exhibits a less sharp attention distribution in this scenario. Further attention visualizations can be found in the accompanying video[2].

E. Robustness

Verifying the robustness of our approach with respect to ICP accuracy against its dependence on static obstacles

[2]https://youtu.be/cYNUFD_rGNE

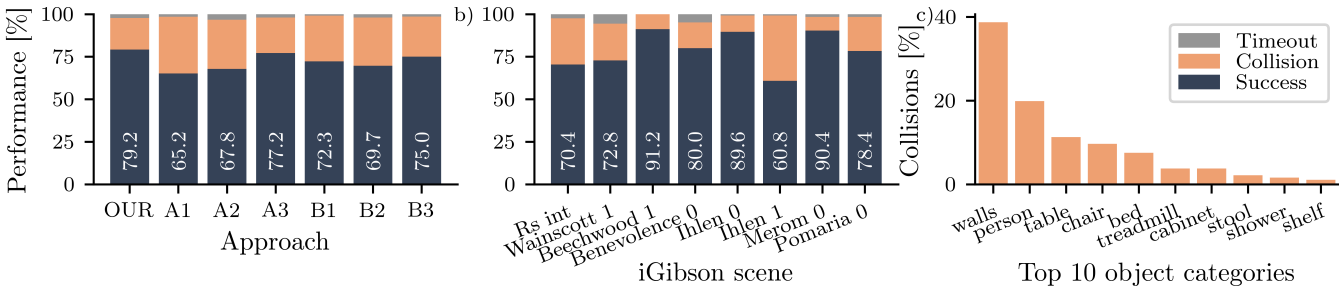


Fig. 7: Results of the generalization study using the iGibson simulator over eight scenes witch 125 episodes each. **a)** OUR controller demonstrates the best generalization capabilities in the sim-to-sim transfer of all approaches. **b)** Breakdown into the different scenes shows a scene-dependency of the controller performance for OUR controller. **c)** Collision object category analysis within iGibson: The most collided-with objects are the walls.

for correct alignment, an open space evaluation of the same evaluation environments but without walls reveals an absolute performance decrease of 3.6%. When disabling ICP alignment entirely and feeding non-aligned lidar scans into the TAGD pipeline, the absolute performance drops by 4.8%. In both cases, the performance is still superior the NO-TEMPORAL ablation, demonstrating decent robustness of the TAGD-based approach against ICP failure in these edge cases. Note that the obstacle parameters of Tab. I were used.

F. Generalization Performance

To investigate the generalization ability of our approach, we evaluated the Pybullet-trained agents in the iGibson simulator [41] in a sim-to-sim transfer, see Fig. 1. The sensor settings and overall navigation objective remain similar, but two major differences strike: 1) The indoor scenarios are of high fidelity with diverse furniture objects and a more complex room architecture. 2) The pedestrians are represented with real 3D meshes instead of cuboids and have a more refined motion simulation. Precisely, we adapt the navigation task from the 2021 iGibson Social Navigation Challenge [42] that features eight scenes and Optimal Reciprocal Collision Avoidance (ORCA) among pedestrians. The key settings to mention as taken over from the original challenge are the maximum pedestrian speed of $0.5\mathrm{m\,s^{-1}}$, an inverse scene area-related population of 8 m^2 per pedestrian, and a goal sample distance between 1.0 and $10.0\mathrm{m}$.

As seen in Fig. 7a), OUR controller exhibits the best generalization performance among all approaches. The slightly lower success rates in Fig. 7a) and Tab. Ic) point towards a simulator gap and increased difficulty within the scenes. Also, the individual scenes seem to be of varying difficulty to the robot, compare Fig. 7b). To further differentiate the challenges the robot faces in the iGibson scenes, the top ten collided-with object categories have been recorded, see Fig. 7c). As the majority of collisions events involve walls, the possibly higher degree of confined spaces within the iGibson scene could play a role. Furthermore, tables and chairs are among the most frequent collision causes. These object are usually thin-legged, providing a challenge for lidar detection at low angular resolutions. In summary, the attention-based architecture surpasses the tested CNN feature extractors in unseen environments.

G. Real-World Experiment

Using the Robot Operating System (ROS) [43], we transferred the trained controller to a real Kobuki Turtlebot 2, as described in Sec. IV-G. In our experiment, the Gmapping package [44], a Simultaneous Localization and Mapping algorithm, was used to build an occupancy grid map of real scenarios upfront for path planning. During navigation, Adaptive Monte Carlo Localization [45] estimated the robot's pose in the pre-mapped environment based on the lidar reading and robot odometry.

We tested our learning-based spatiotemporal approach qualitatively in various real-world scenarios, including corridors, intersections, and offices. Please refer to our supplemental video[2] of the real-world experiment. In a corridor, the two participants overtake the robot from behind or approach it rigorously from the front, see Fig. 1. The robot smoothly gives room to the pedestrians and avoids collision. At an intersection, pedestrians appear from the blind spots behind a corner. In another test the pedestrian blocks the doorway to see whether the robot would stop upon facing the impassable situation. All navigation situations are successfully handled by our spatiotemporal controller.

VI. Conclusions

We proposed a novel and lightweight approach for robot navigation in dynamic indoor environments. Our learning-based approach featuring spatiotemporal attention demonstrates the capacity to highlight collision-relevant features from the sensor data, making the most out of the sparse 2D lidar readings. Meanwhile, the introduced temporal accumulation group descriptors (TAGD) help to counteract the robot self-movement over subsequent lidar readings and therefore support the differentiation between static and dynamic obstacles without explicit object tracking. Our policy directly outputs linear and angular velocity, leading to smooth robot navigation, and outperforms several state-of-the-art approaches in terms of collision rate for different pedestrian speed and number of obstacles. We validate the sim-to-sim generalization capabilities in the iGibson simulator, finding excellent and better than state-of-the-art performance to unseen, more complex indoor environments with different pedestrian dynamics. Lastly, we achieve an effortless sim-to-real transfer into dynamic real-world indoor environments.

References

[1] K. D. Katyal, G. D. Hager, and C.-M. Huang, "Intent-aware pedestrian prediction for adaptive crowd navigation," in *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*. IEEE, 2020.

[2] A. Wang, C. Mavrogiannis, and A. Steinfeld, "Group-based motion prediction for navigation in crowded environments," in *Conference on Robot Learning*. PMLR, 2022.

[3] Y. Wang, Q. Mao, H. Zhu, J. Deng, Y. Zhang, J. Ji, H. Li, and Y. Zhang, "Multi-modal 3d object detection in autonomous driving: a survey," *International Journal of Computer Vision*, 2023.

[4] C. Wang, C. Ma, M. Zhu, and X. Yang, "Pointaugmenting: Cross-modal augmentation for 3d object detection," in *Proc. of the IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2021.

[5] J. Li, H. Qin, J. Wang, and J. Li, "Openstreetmap-based autonomous navigation for the four wheel-legged robot via 3d-lidar and ccd camera," *IEEE Trans. Ind. Electron.*, vol. 69, no. 3, 2021.

[6] M. Himmelsbach, F. V. Hundelshausen, and H.-J. Wuensche, "Fast segmentation of 3d point clouds for ground vehicles," in *2010 IEEE Intelligent Vehicles Symposium*. IEEE, 2010.

[7] C. Chen, Y. Liu, S. Kreiss, and A. Alahi, "Crowd-robot interaction: Crowd-aware robot navigation with attention-based deep reinforcement learning," in *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*. IEEE, 2019.

[8] C. Chen, S. Hu, P. Nikdel, G. Mori, and M. Savva, "Relational graph learning for crowd navigation," in *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*. IEEE, 2020.

[9] E. Salvato, G. Fenu, E. Medvet, and F. A. Pellegrino, "Crossing the Reality Gap: A Survey on Sim-to-Real Transferability of Robot Controllers in Reinforcement Learning," *IEEE Access*, vol. 9, 2021.

[10] X. Chen, J. Hu, C. Jin, L. Li, and L. Wang, "Understanding Domain Randomization For Sim-To-Real Transfer," in *10th International Conference on Learning Representations, ICLR 2022*, 2022.

[11] X. Xiao, B. Liu, G. Warnell, and P. Stone, "Motion planning and control for mobile robot navigation using machine learning: A survey," *Autonomous Robots*, vol. 46, no. 5, Jun. 2022.

[12] L. Tai, G. Paolo, and M. Liu, "Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation," in *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*. IEEE, 2017.

[13] B. Qin, Z. J. Chong, S. H. Soh, T. Bandyopadhyay, M. H. Ang, E. Frazzoli, and D. Rus, "A spatial-temporal approach for moving object recognition with 2d lidar," in *Experimental Robotics: The 14th International Symposium on Experimental Robotics*. Springer, 2016.

[14] Y. Song, Y. Tian, G. Wang, and M. Li, "2d lidar map prediction via estimating motion flow with gru," in *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*. IEEE, 2019.

[15] C. Pérez-D'Arpino, C. Liu, P. Goebel, R. Martín-Martín, and S. Savarese, "Robot Navigation in Constrained Pedestrian Environments using Reinforcement Learning," in *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*, May 2021.

[16] J. Liang, U. Patel, A. J. Sathyamoorthy, and D. Manocha, "Realtime Collision Avoidance for Mobile Robots in Dense Crowds using Implicit Multi-sensor Fusion and Deep Reinforcement Learning," *arXiv:2004.03089 [cs]*, Apr. 2020.

[17] J. de Heuvel, W. Shi, X. Zeng, and M. Bennewitz, "Subgoal-Driven Navigation in Dynamic Environments Using Attention-Based Deep Reinforcement Learning," *arXiv:2303.01443 [cs]*, Mar. 2023.

[18] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, 1997.

[19] O. Brock and O. Khatib, "High-speed navigation using the global dynamic window approach," in *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*, vol. 1. IEEE, 1999.

[20] M. Missura and M. Bennewitz, "Predictive Collision Avoidance for the Dynamic Window Approach," in *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*, May 2019.

[21] X. Xiao, B. Liu, G. Warnell, J. Fink, and P. Stone, "Appld: Adaptive planner parameter learning from demonstration," *IEEE Robotics and Automation Letters (RA-L)*, vol. 5, no. 3, 2020.

[22] J. Shabbir and T. Anwer, "A survey of deep learning techniques for mobile robot applications," *arXiv preprint arXiv:1803.07608*, 2018.

[23] H.-T. L. Chiang, A. Faust, M. Fiser, and A. Francis, "Learning navigation behaviors end-to-end with autorl," *IEEE Robotics and Automation Letters (RA-L)*, vol. 4, no. 2, 2019.

[24] J. de Heuvel, N. Corral, B. Kreis, J. Conradi, A. Driemel, and M. Bennewitz, "Learning Depth Vision-Based Personalized Robot Navigation From Dynamic Demonstrations in Virtual Reality," in *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*, Oct. 2023.

[25] A. Faust, K. Oslund, O. Ramirez, A. Francis, L. Tapia, M. Fiser, and J. Davidson, "Prm-rl: Long-range robotic navigation tasks by combining reinforcement learning and sampling-based planning," in *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*. IEEE, 2018.

[26] A. Bayoumi and M. Bennewitz, "Learning optimal navigation actions for foresighted robot behavior during assistance tasks," in *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*. IEEE, 2016.

[27] A. Pfrunder, P. V. Borges, A. R. Romero, G. Catt, and A. Elfes, "Real-time autonomous ground vehicle navigation in heterogeneous environments using a 3d lidar," in *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*. IEEE, 2017.

[28] T. Fan, X. Cheng, J. Pan, P. Long, W. Liu, R. Yang, and D. Manocha, "Getting robots unfrozen and unlost in dense pedestrian crowds," *IEEE Robotics and Automation Letters (RA-L)*, vol. 4, no. 2, 2019.

[29] R. Q. Charles, H. Su, M. Kaichun, and L. J. Guibas, "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *Proc. of the IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*. IEEE, Jul. 2017.

[30] Z. Ling, Y. Yao, X. Li, and H. Su, "On the Efficacy of 3D Point Cloud Reinforcement Learning," *arXiv:2306.06799 [cs]*, Jun. 2023.

[31] S. Kraemer, C. Stiller, and M. E. Bouzouraa, "LiDAR-Based Object Tracking and Shape Estimation Using Polylines and Free-Space Information," in *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*, Oct. 2018.

[32] H. Chen and P. Lu, "Real-time Identification and Simultaneous Avoidance of Static and Dynamic Obstacles on Point Cloud for UAVs Navigation," *arXiv:2110.10360 [cs]*, Oct. 2021.

[33] H. Zhao, L. Jiang, J. Jia, P. H. S. Torr, and V. Koltun, "Point Transformer," in *Proc. of the IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2021.

[34] K. S. Arun, T. S. Huang, and S. D. Blostein, "Least-squares fitting of two 3-d point sets," *IEEE Transactions on pattern analysis and machine intelligence*, no. 5, 1987.

[35] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.

[36] Z. Niu, G. Zhong, and H. Yu, "A review on the attention mechanism of deep learning," *Neurocomputing*, vol. 452, Sep. 2021.

[37] T. Luong, H. Pham, and C. D. Manning, "Effective Approaches to Attention-based Neural Machine Translation," in *Proc. of the 2015 Conf. on Empirical Methods in Natural Language Processing*, L. Màrquez, C. Callison-Burch, and J. Su, Eds. Association for Computational Linguistics, Sep. 2015.

[38] E. Coumans and Y. Bai, "Pybullet, a python module for physics simulation for games, robotics and machine learning," http://pybullet.org, 2016–2019.

[39] A. Francis, C. Perez-D'Arpino, C. Li, F. Xia, A. Alahi, R. Alami, A. Bera, A. Biswas, J. Biswas, R. Chandra, H.-T. L. Chiang, M. Everett, S. Ha, J. Hart, J. P. How, H. Karnan, T.-W. E. Lee, L. J. Manso, R. Mirksy, S. Pirk, P. T. Singamaneni, P. Stone, A. V. Taylor, P. Trautman, N. Tsoi, M. Vazquez, X. Xiao, P. Xu, N. Yokoyama, A. Toshev, and R. Martin-Martin, "Principles and Guidelines for Evaluating Social Robot Navigation Algorithms," *arXiv:2306.16740 [cs]*, Jun. 2023.

[40] H. Kolivand, M. S. Rahim, M. S. Sunar, A. Z. A. Fata, and C. Wren, "An integration of enhanced social force and crowd control models for high-density crowd simulation," *Neural Computing and Applications*, vol. 33, no. 11, Jun. 2021.

[41] B. Shen, F. Xia, C. Li, R. Martín-Martín, L. Fan, G. Wang, C. Pérez-D'Arpino, S. Buch, S. Srivastava, L. Tchapmi, M. Tchapmi, K. Vainio, J. Wong, L. Fei-Fei, and S. Savarese, "iGibson 1.0: A Simulation Environment for Interactive Tasks in Large Realistic Scenes," in *Proc. of the IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS)*, Sep. 2021.

[42] "iGibson Challenge 2021." [Online]. Available: https://svl.stanford.edu/igibson/challenge2021.html

[43] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, "ROS: An open-source Robot Operating System," in *ICRA Workshop on Open Source Software*, vol. 3. Kobe, Japan, 2009.

[44] G. Grisetti, C. Stachniss, and W. Burgard, "Improving grid-based slam with rao-blackwellized particle filters by adaptive proposals and selective resampling," in *Proc. of the IEEE Intl. Conf. on Robotics & Automation (ICRA)*. IEEE, 2005.

[45] D. Fox, W. Burgard, F. Dellaert, and S. Thrun, "Monte carlo localization: Efficient position estimation for mobile robots," *Aaai/iaai*, vol. 1999, no. 343-349, 1999.